

Contents

I	Introduction	2
1	Background Knowledge	3
1.1	Markov Decision Process	3
1.2	Useful Facts	3
1.3	Glossaries	3
2	Policy Gradient	4
2.1	Policy Gradient Estimator	4
2.2	REINFORCE algorithm	5
2.3	Actor-Critic Algorithm	5
II	Algorithms	7
3	Proximal Policy Optimization	8
3.1	PPO introduced by OpenAI	8
3.2	PPO in the context of RLHF	9
4	REINFORCE++	11
5	Implicit Reward in Direct Preference Optimization	13
5.1	DPO Objective	13
5.2	Reward Hacking in DPO	13
5.3	Common Failure Modes of DPO	14
5.4	Mitigating Reward Hacking and Over-Optimization in DPO	14
6	Group Relative Policy Optimization	16
7	Group Sequence Policy Optimization	17
8	Clipped Importance Sampling Policy Optimization	19
8.1	Gradient Difference between PPO and CISPO	19
III	Asynchronous RL	21
9	SAO	23
9.1	Algorithm	23
9.2	Guessed Advantage Estimator	24
9.3	Practical Considerations	26

Part I

Introduction

1 Background Knowledge

1.1 Markov Decision Process

Definition 1.1 (State, Action, Trajectory). A sequence of actions and states with T interactions with environment is defined as a trajectory $\tau = (s_0, a_0, s_1, a_1, s_2, a_2, \dots, a_{T-1}, s_{T-1})$

Definition 1.2 (Reward and Return). At time step t , the agent observes state s_t , takes action a_t , and transitions to the next state s_{t+1} . The single-step reward is determined by the transition and is defined as

$$r_t = R(s_t, a_t, s_{t+1}).$$

The (discounted) **reward** on the trajectory τ is defined as

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t,$$

where $\gamma \in (0, 1]$ is the discount factor. The discounted **return** starting from time step t is defined as

$$G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}.$$

Equivalently, G_t satisfies the recursive relation

$$G_t = r_t + \gamma G_{t+1}.$$

Definition 1.3 (State-Value Function and Value Function). The value function for the policy π is defined as

$$\begin{aligned} V^\pi(s_t) &= \mathbb{E}_\pi[G_t | s_t] = \mathbb{E}_\pi[r_t + \gamma G_{t+1} | s_t] = \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma \mathbb{E}_\pi[G_{t+1} | s_{t+1}]] \\ &= \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma V^\pi(s_{t+1})]. \end{aligned}$$

The state-value function for the policy π is defined as

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi[G_t | s_t, a_t] = \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma V^\pi(s_{t+1})].$$

Therefore,

$$V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} [Q^\pi(s_t, a_t)]$$

Definition 1.4 (Advantage). The advantage function, by its definition, measures whether or not the action is better or worse than the policy's default behavior.

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) = \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} \left[\underbrace{r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)}_{\text{temporal difference}} \right] \quad (1)$$

1.2 Useful Facts

1. Probability of a trajectory

$$P(\tau | \theta) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} | s_t, a_t) \pi_\theta(a_t | s_t) \quad (2)$$

2. log derivative trick

$$\nabla_\theta P(\tau | \theta) = P(\tau | \theta) \nabla_\theta \log P(\tau | \theta) \quad (3)$$

3. expected Grad-Log-Prob Lemma

$$\mathbb{E}_{x \sim p(\cdot; \theta)} [\nabla_\theta \log p(x; \theta)] = 0 \quad (4)$$

1.3 Glossaries

Definition 1.5 (Estimation of the KL divergence). There are different estimators of the KL divergence, which can be found here. Recall that $\text{KL}(q||p) = \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right]$,

- k_1 estimator: $\frac{1}{n} \sum_{i=1}^n \log \frac{q(x_i)}{p(x_i)}$. Unbiased, high variance. Can be negative.
- k_2 estimator: $\frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\log \frac{p(x_i)}{q(x_i)} \right)^2$. biased, low variance. Always positive.
- k_3 estimator: $\frac{p(x)}{q(x)} - 1 - \log \frac{p(x)}{q(x)}$. Always positive. Better than k_2 ? (non-negativity: $\log a \leq a - 1$ for all $a > 0$).

vd: To add: comparison with three estimators

2 Policy Gradient

The goal of the policy-based RL is to maximize the expected reward

$$\max \left\{ \mathcal{J}(\theta) := \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \int P(\tau|\theta) R(\tau) d\tau \right\}, \quad (5)$$

where $P(\tau|\theta)$ is defined in (2) and $s_0 \sim \rho_0(\cdot)$

- $a_t \sim \pi_\theta(\cdot|s_t)$ for all $t \in \{0, 1, \dots, T-1\}$.
- $s_{t+1} \sim P(\cdot|s_t, a_t)$ for all $t \in \{0, 1, \dots, T-1\}$.
- $r_t = R(s_t, a_t, s_{t+1})$ for all $t \in \{0, 1, \dots, T-1\}$.

Applying (2) and (3), we get

$$\begin{aligned} \nabla_\theta \mathcal{J}(\theta) &= \int \nabla_\theta P(\tau|\theta) R(\tau) d\tau = \int P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \left(\rho_0(s_0) + \sum_{t=0}^{T-1} P(s_{t+1}|s_t, a_t) \right) + \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] R(\tau) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \end{aligned} \quad (6)$$

2.1 Policy Gradient Estimator

Given N trajectories $\{\tau_1, \dots, \tau_N\}$ with $\tau_n \sim \pi_\theta$ for all $n \in [N]$, how can we construct a "good" policy gradient estimator?

The natural choice would be

- Naive unbiased estimator

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} R(\tau_n) \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

However, the above looks a bit weird in the sense that $\log \pi_\theta(a_t|s_t)$ contains single-step probability while $R(\tau_n)$ represents the whole trajectory. This naturally leads to the following abstraction

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} \Phi_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

Some popular choices of Φ_t (Schulman et al., 2015), hence the gradient estimator can be

- Reward-to-go unbiased estimator (drops a_t 's dependency on the past actions)

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} G_t(\tau_n) \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

Recall $G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$. If one adopts this estimator, then the algorithm is called **REINFORCE** algorithm. This method drops the need on the Value function (see details below) but suffers from the high variance in gradient estimates.

- with baseline (variance reduction)

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} (G_t(\tau_n) - b(s_t)) \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

One popular choice of the baseline function is $b(s_t) := V^\pi(s_t)$. When V^π is unknown, we can approximate it with $V_\phi^\pi(\cdot)$, a neural network parameterized by ϕ .

- Advantages based estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$.

- TD-error based estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left((r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)) \sum_{t=0}^{T_n-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$ is a approximation of the $A^{\pi}(s_t, a_t)$.

- GAE(γ, λ) estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{\ell=0}^{T_n} (\gamma \lambda)^{\ell} \delta_{t+\ell} \sum_{t=0}^{T_n-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $\delta_t = r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t)$. GAE($\gamma, 0$) typically has low variance and GAE($\gamma, 1$) has high variance.

2.2 REINFORCE algorithm

If one adopts the Reward-to-go unbiased estimator, we can reach to the REINFORCE algorithm. The objective function is

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left(\sum_{t=0}^{T-1} G_t(\tau) \log \pi_{\theta}(a_t | s_t) \right) \quad (7)$$

where $G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$. This method drops the need on the Value function but suffers from the high variance in gradient estimates. The algorithm is summarized in Algorithm 1

Algorithm 1 REINFORCE

- 1: Inputs: Initial Policy network parameter θ_0 and stepsize α .
- 2: **for** $k=0,1,2,\dots$ **do**
- 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
- 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r_{t'}$ for all n .
- 5: Compute the gradient estimate

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta_k)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} G_t(\tau_n) \nabla_{\theta} \log \pi_{\theta_k}(a_t | s_t) \right)$$

- 6: Policy update $\theta_{k+1} \leftarrow \theta_k + \alpha \widetilde{\nabla_{\theta} \mathcal{J}(\theta_k)}$
 - 7: **end for**
-

2.3 Actor-Critic Algorithm

When one adopts the estimator for the policy gradient, usually the state value function is used. Let us approximate it with V_{ϕ}^{π} the objective function for TD-residual based gradient estimator can written as

$$\max \left\{ \mathcal{J}_{\text{TD}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} (r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t)) \log \pi_{\theta}(a_t | s_t) \right] \right\}.$$

Similarly, the V_{ϕ}^{π} can be updated by

$$\min \left\{ \mathcal{L}(\phi) := \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} [(r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t))^2] \right] \right\}. \quad (8)$$

This is actually the actor-critic framework. When we drive $\mathcal{L}(\phi)$ to 0, intuitively, the advantage will be close to 0, which means that for the actor, π_{θ} is approaching the optimal policy and the value function is also approaching the optimal.

vd:To add

Algorithm 2 Naive Actor-Critic Algorithm

- 1: Inputs: Initial Policy network parameter θ_0 and stepsize α .
 - 2: **for** $k=0,1,2,\dots$ **do**
 - 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
 - 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r'_{t'}$ for all n .
 - 5: Compute the gradient estimate
 - 6: Policy update $\theta_{k+1} \leftarrow \theta_k + \alpha \widetilde{\nabla_{\theta} \mathcal{J}(\theta_k)}$
 - 7: **end for**
-

Part II

Algorithms

3 Proximal Policy Optimization

3.1 PPO introduced by OpenAI

When performing the policy gradient update, each step one needs to sample the trajectories of the environment (since after one update on the policy π_θ , the trajectory becomes obsolete. How can we address the sample efficiency issue? Importance sampling comes to rescue.

Consider the gradient estimator function, where $\hat{A}_\phi$ is any "good" estimate of the advantage¹.

$$\begin{aligned}\widetilde{\nabla_\theta \mathcal{J}}(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \hat{A}_\phi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{1}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \nabla_\theta \pi_\theta(a_t | s_t) \right].\end{aligned}$$

The corresponding objective function can be written as

$$\mathcal{J}_{\text{advantage}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \right]$$

In practice, we cannot sample enough trajectories, hence the estimation of the $\mathcal{J}_{\text{advantage}}(\theta)$ through sample average is not accurate. This will be worsen by the fact that if $\pi_{\theta_{\text{old}}}$ is very different from π_θ . This leads to two versions of the PPO objective functions.

$$\max_{\theta} \mathcal{J}_{\text{PPO-clip}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \min \left\{ \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t), \text{clip} \left(\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_\phi(s_t, a_t) \right\} \right], \quad (9)$$

where

$$\text{clip}(x, 1 - \epsilon, 1 + \epsilon) = \begin{cases} x, & \text{if } 1 - \epsilon < x < 1 + \epsilon \\ 1 - \epsilon, & \text{if } x \leq 1 - \epsilon \\ 1 + \epsilon, & \text{if } x \geq 1 + \epsilon. \end{cases}$$

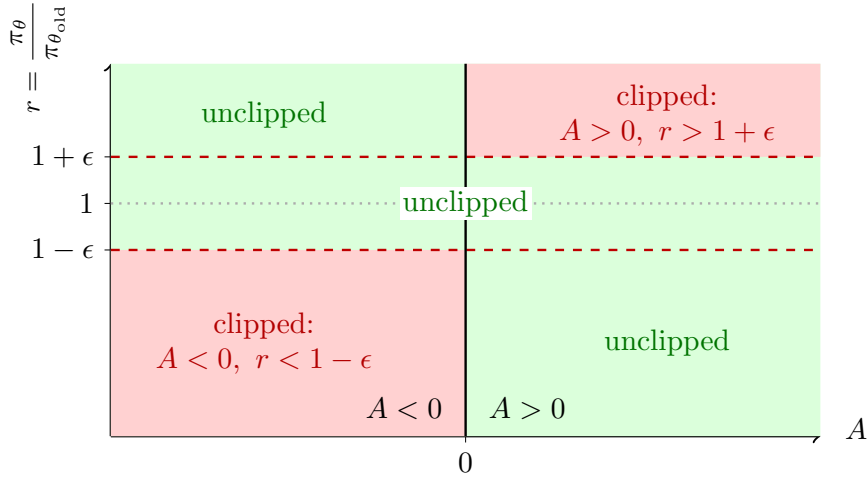


Figure 1: PPO-Clip trust region in the (A, r) plane, where $r = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. The red region is the clipped region, and the green region is the unclipped region (after the clip and min operation).

Remark

- The popular choice of $\hat{A}_\phi(s_t, a_t)$ is $A_\phi^{\text{GAE}(\gamma, \lambda)} = \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell \left(r_{t+\ell} + \gamma V_\phi^\pi(s_{t+1+\ell}) - V_\phi^\pi(s_{t+\ell}) \right)$.

¹As the subscripts suggests, the definition of $\hat{A}_\phi$ usually rely on the value function V_ϕ^π .

- When $\hat{A}_\phi(s_t, a_t) > 0$, which means that a_t is better than the average, we want to increase its probability. However, we need to limit the increase to not being greater than the factor of $1 + \epsilon$.
- When $\hat{A}_\phi(s_t, a_t) < 0$, which means that a_t is worse than the average, we want to decrease its probability. However, we need to limit the decrease to be no greater than the factor of $1 - \epsilon$.

One can also constraint the distribution shift by

$$\max_{\theta} \mathcal{J}_{\text{PPO-KL}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_{\phi}(s_t, a_t) - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t) \|\pi_{\theta}(\cdot|s_t)] \right], \quad (10)$$

Remark

- When maximize $\mathcal{J}_{\text{PPO-KL}}(\theta)$, we minimize $\mathbb{D}_{\text{KL}}[\pi_{\theta_{\text{old}}}(\cdot|s_t) \|\pi_{\theta}(\cdot|s_t)]$, i.e., we minimize the forward KL, which has the mean-seeking behavior.

To put pieces together, we summarize the PPO-Clip in Algorithm 3.

Algorithm 3 PPO-Clip Algorithm

- 1: Inputs: Initial Policy network parameter θ_0 and value function parameter ϕ_0 .
- 2: **for** $k=0,1,2,\dots$ **do**
- 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
- 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r_{t'}$ for all n .
- 5: GAE calculation $A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)})$ for all t and n .
- 6: Approximately solve

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_{\theta}(a_t^{(n)}|s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)}|s_t^{(n)})} A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)}), \right. \\ \left. \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)}) \right\},$$

- 7: Approximately solve

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} (V_{\phi}^{\pi}(s_t^{(n)}) - G_t(\tau_n))^2$$

- 8: **end for**
-

3.2 PPO in the context of RLHF

Let us first introduce a new set of notions and setup. Let $\mathcal{D}$ be a set of prompt and $x \in \mathcal{D}$ is any sample from it. $y \sim \pi_{\theta}(\cdot|x)$ be a response sampled from the policy model π_{θ} given the prompt x . To simplify the discussion, let us assume that the reward model is given and we focus on the outcome reward $r(x, y)$.

We consider the following variant objective function inspired by policy gradient and PPO

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r(x, y) - \beta \text{KL}[\pi_{\theta}(\cdot|x) \|\pi_{\theta_{\text{ref}}}(\cdot|x)]] ,$$

note that here we used forward KL divergence. Then the gradient is

$$\nabla \mathcal{J}(\theta) = \int_{x \in \mathcal{X}} p(x) \left\{ \left[\int_{y \in \mathcal{Y}(x)} r(x, y) \frac{\partial}{\partial \theta} \pi_{\theta}(y|x) dy \right] - \beta \left[\int_{s \in \mathcal{Y}(x)} \frac{\partial}{\partial \theta} \pi_{\theta}(s|x) \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \right] \right\} dx, \quad (11)$$

where $\mathcal{X} = \mathcal{D}$ and $\mathcal{Y}(x) = \text{support}(\pi_{\theta}(y|x))$. Note that

$$\begin{aligned} & \int_{s \in \mathcal{Y}(x)} \frac{\partial}{\partial \theta} \pi_{\theta}(s|x) \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \\ &= \int_{s \in \mathcal{Y}(x)} \left[\nabla \pi_{\theta}(s|x) \cdot \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} + \pi_{\theta}(s|x) \cdot \nabla \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} \right] ds \\ &= \int_{s \in \mathcal{Y}(x)} \left[\nabla \pi_{\theta}(s|x) \cdot \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} + \nabla \pi_{\theta}(s|x) \right] ds \\ &= \int_{s \in \mathcal{Y}(x)} \nabla \pi_{\theta}(s|x) \cdot \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \end{aligned} \quad (12)$$

where the last line follows from (4). Combining (11) and (12), one obtains

$$\begin{aligned}
\nabla \mathcal{J}(\theta) &= \int_{x \in \mathcal{X}} p(x) \left\{ \left[\int_{y \in \mathcal{Y}(x)} r(x, y) \frac{\partial}{\partial \theta} \pi_{\theta}(y|x) dy \right] - \beta \left[\int_{s \in \mathcal{Y}(x)} \nabla \pi_{\theta}(s|x) \cdot \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \right] \right\} dx \\
&= \int_{x \in \mathcal{X}, y \in \mathcal{Y}(x)} p(x) \left\{ \pi_{\theta}(y|x) \left[r(x, y) \nabla \log \pi_{\theta}(y|x) - \beta \nabla \log \pi_{\theta}(y|x) \cdot \log \frac{\pi_{\theta}(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right] \right\} dy dx \\
&= \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} \left[\left(r(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right) \nabla \log \pi_{\theta}(y|x) \right] \tag{13}
\end{aligned}$$

From (13), we conclude the "PPO" version of the RLHF objective function

$$\max_{\theta} \mathcal{J}_{\text{RLHF}}(\theta) := \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} \left[r(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right], \tag{14}$$

which recovers the first part of the objective function in Equation (2) of Ouyang et al. (2022).

4 REINFORCE++

REINFORCE++ modifies PPO in the LLM text by introducing

- token-level KL penalty $\text{KL}(t; \theta) = \log \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$, where $\pi_{\theta_{\text{ref}}}$ can be the SFT model;
- token-level reward function $r_t = \mathbb{I}(s_t = [\text{EOS}])r(x, y) - \beta \text{KL}(t)$, where x and y are input prompt and response, respectively and $r(x, y)$ is the outcome reward.

Recall the advantage used in PPO is $A_\phi^{\text{GAE}(\gamma, \lambda)} = \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell \left(r_{t+\ell} + \gamma V_\phi^\pi(s_{t+1+\ell}) - V_\phi^\pi(s_{t+\ell}) \right)$. If we drop the value network and plug in the new definition of r_t , we get

$$\begin{aligned} A_t^{\text{GAE}(\gamma, \lambda)} &= \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell [\mathbb{I}(s_{t+\ell} = [\text{EOS}])r(x, y) - \beta \text{KL}(t + \ell; \theta)] \\ &= \sum_{i=t}^{T-1} (\gamma\lambda)^\ell [\mathbb{I}(s_i = [\text{EOS}])r(x, y) - \beta \text{KL}(i)] = (\gamma\lambda)^{T-1} r(x, y) - \sum_{i=t}^{T-1} (\gamma\lambda)^i [\beta \text{KL}(i; \theta)]. \end{aligned}$$

Specifically, letting $\gamma = \lambda = 1$, we get the advantage estimation used in REINFORCE++ as

$$A_t^{\text{RF}++} = r(x, y) - \sum_{i=t}^{T-1} \beta \text{KL}(i; \theta). \quad (15)$$

Consequently, for the k th episode, REINFORCE++ solves the following problem

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_\theta(a_t^{(n)}|s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)}|s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} \right\},$$

where $\bar{A}_{n,t} = \frac{A_{n,t}^{\text{RF}++} - \mu_k}{\sigma_k}$, $\mu_k = \text{mean}\{A_{n,t}^{\text{RF}++}\}_{n \in [1, N], t \in [0, T_n-1]}$, $\sigma_k = \text{std}\{A_{n,t}^{\text{RF}++}\}_{n \in [1, N], t \in [0, T_n-1]}$, $A_{n,t}^{\text{RF}++} = r(x^{(n)}, y^{(n)}) - \sum_{i=t}^{T_n-1} \beta \text{KL}(i; \theta_k)$, and $(x^{(n)}, y^{(n)})$ is the n th sample.

To conclude, we summarize REINFORCE++ in Algorithm 4.

Algorithm 4 REINFORCE++

- 1: Inputs: Initial Policy network parameter θ_0 .
- 2: **for** $k=0, 1, 2, \dots$ **do**
- 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
- 4: $\{A_{n,t}^{\text{RF}++}\}_{n \in [1, N], t \in [0, T_n-1]}$ calculation, batch mean μ_k and variance σ_k calculation.
- 5: Compute normalized advantage estimation $\{\bar{A}_{n,t} = \frac{A_{n,t}^{\text{RF}++} - \mu_k}{\sigma_k}\}_{n \in [1, N], t \in [0, T_n-1]}$.
- 6: Approximately solve

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_\theta(a_t^{(n)}|s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)}|s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} \right\},$$

7: **end for**

Remark

- Compared to GRPO, the normalization happens on the "batch" level; while GRPO applies the normalization within the "group" level.
- The KL penalty applied in the loss function is different. For GRPO, the KL penalty is taken out of the definition of the advantage estimation and uses the k_3 estimator (recall the definition in Definition 1.5), which is $\beta \mathbb{D}_{\text{KL}}[\pi_\theta || \pi_{\text{ref}}](i, t)$. In REINFORCE++, the KL penalty is factorized in the definition of the advantage estimation and uses the k_1 estimator, which is $\beta \sum_{i=t}^{T-1} \text{KL}(i; \theta)$. Notice that in the t th action, the penalty is the accumulation of all KL penalties starting from the time step t to the end.

Motivated by the discussion above, Xie et al. (2025) modifies the loss function by combining the GRPO and REINFORCE++ as below.

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_{\theta}(a_t^{(n)} | s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)} | s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{\text{ref}}] (i, t) \right\},$$

where $\bar{A}_{n,t} = \frac{A_{n,t}^{\text{Logic}} - \mu_k}{\sigma_k}$, $\mu_k = \text{mean}\{A_{n,t}^{\text{Logic}}\}_{n \in [1, N], t \in [0, T_n - 1]}$, $\sigma_k = \text{std}\{A_{n,t}^{\text{Logic}}\}_{n \in [1, N], t \in [0, T_n - 1]}$, $A_{n,t}^{\text{Logic}} = r(x^{(n)}, y^{(n)})$. Basically, it moves KL penalty out of the definition of the advantage estimators and use k_3 KL estimation.

5 Implicit Reward in Direct Preference Optimization

Direct Preference Optimization (DPO) does not explicitly train a separate reward model. Instead, it implicitly defines a reward function through the relationship between the optimized policy and the reference policy. DPO can be derived from the KL-regularized RLHF objective:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)),$$

where π_{ref} is a fixed reference policy, usually the supervised fine-tuned model, and $\beta > 0$ controls the strength of the KL regularization.

The optimal policy of this objective satisfies

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

where $Z(x)$ is the partition function depending only on the prompt x .

Solving for the reward gives

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x).$$

Since $\beta \log Z(x)$ depends only on the prompt x , it cancels when comparing two responses to the same prompt. Therefore, DPO uses the following implicit reward parameterization:

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} + C(x),$$

where $C(x)$ is a prompt-dependent constant.

For a preferred response y_w and a rejected response y_l , the reward difference is

$$\begin{aligned} r_{\theta}(x, y_w) - r_{\theta}(x, y_l) &= \beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \\ &= \beta [\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_w|x) + \log \pi_{\text{ref}}(y_l|x)]. \end{aligned}$$

Thus, the implicit reward in DPO measures how much the optimized policy increases the probability of a response relative to the reference policy.

5.1 DPO Objective

DPO is trained on preference data of the form

$$(x, y_w, y_l) \sim \mathcal{D},$$

where x is the prompt, y_w is the preferred response, and y_l is the rejected response.

The DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right) \right],$$

where $\sigma(\cdot)$ is the sigmoid function.

Define

$$\Delta_{\theta}(x, y_w, y_l) = \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)}.$$

Then the DPO loss can be written compactly as

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(\beta \Delta_{\theta}(x, y_w, y_l))].$$

Minimizing this loss increases the relative log-probability of the preferred response over the rejected response, after correcting for their probabilities under the reference policy.

5.2 Reward Hacking in DPO

DPO does not use an explicit reward model during optimization. Therefore, it is less likely to suffer from the classical online reward hacking problem in PPO-style RLHF, where the policy repeatedly exploits weaknesses of a learned reward model.

However, DPO can still suffer from a different form of reward hacking or over-optimization. Since the implicit reward is determined by the preference dataset,

$$(x, y_w, y_l) \sim \mathcal{D},$$

DPO may exploit biases or artifacts in the preference data.

In this sense, DPO does not hack an explicit reward model. Instead, it may overfit or exploit the implicit preference signal encoded in the dataset:

DPO may hack the preference dataset rather than a learned reward model.

For example, if preferred responses are systematically longer than rejected responses, the model may learn the spurious rule

longer response $\Rightarrow$ better response.

If preferred responses often contain certain stylistic patterns, the model may learn to imitate those patterns even when they do not improve semantic quality.

Thus, DPO can still exhibit reward-hacking-like behavior through preference-data bias, distribution shift, or over-optimization of the implicit reward.

5.3 Common Failure Modes of DPO

A common failure mode of DPO is length bias. The sequence log-probability is

$$\log \pi_{\theta}(y \mid x) = \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

If preferred responses are usually longer than rejected responses, the model may learn to favor longer responses, even when they are not more accurate or helpful.

Another failure mode is style bias. If preferred responses share superficial stylistic features, such as excessive politeness, rigid formatting, or repeated templates, DPO may learn these features as proxies for quality.

DPO may also overfit to easy negatives. If the rejected response y_l is much worse than the preferred response y_w , the model only learns a coarse distinction:

$$y_w \gg y_l.$$

This may not teach the model how to distinguish between two high-quality responses.

Finally, DPO is an offline preference optimization method. It is trained on fixed preference data, but after training, the model generates responses from its own policy:

$$y \sim \pi_{\theta}(\cdot \mid x).$$

If this generation distribution differs significantly from the training preference data distribution, DPO may suffer from distribution shift.

5.4 Mitigating Reward Hacking and Over-Optimization in DPO

One important mitigation is improving the quality of the preference data. The preference dataset should minimize spurious correlations between preference labels and superficial features such as length, format, or style. Ideally, the preference label should reflect semantic quality rather than dataset artifacts.

To reduce length bias, one can use length-normalized log-probabilities. Instead of using

$$\log \pi_{\theta}(y \mid x) = \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}),$$

one may use the average log-probability

$$\frac{1}{|y|} \log \pi_{\theta}(y \mid x) = \frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t \mid x, y_{<t}).$$

Another mitigation is tuning the temperature parameter β . The parameter β controls the strength of the preference margin in the DPO loss:

$$\log \sigma(\beta \Delta_{\theta}(x, y_w, y_l)).$$

If β is too large, the optimization may aggressively separate preferred and rejected responses, increasing the risk of over-optimization. A smaller β makes the update more conservative.

One can also monitor or explicitly regularize the divergence from the reference model:

$$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}).$$

Although DPO already uses the reference policy in its objective, explicit KL monitoring or early stopping can help prevent excessive drift from the reference model.

Another practical method is to mix the DPO loss with a supervised fine-tuning loss:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{DPO}}(\theta) + \lambda_{\text{SFT}} \mathcal{L}_{\text{SFT}}(\theta),$$

where

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x, y_w) \sim \mathcal{D}} [\log \pi_{\theta}(y_w \mid x)].$$

The SFT loss helps preserve language quality, instruction-following behavior, and formatting stability.

Finally, iterative preference data collection can reduce distribution shift. Instead of training only on a fixed offline dataset, one can repeatedly sample responses from the current policy,

$$y \sim \pi_{\theta}(\cdot \mid x),$$

collect new preference labels, and continue preference optimization on data closer to the current model distribution.

6 Group Relative Policy Optimization

GRPO is motivated by removing the requirements on estimating the value function is used PPO Algorithm 3. For each question $q \sim P(Q)$, GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$.

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta}||\pi_{\text{ref}}](i, t) \right\} \right], \quad (16)$$

where

- $\mathbb{D}_{KL}[\pi_{\theta}||\pi_{\text{ref}}]$ is the k_3 estimator of the KL divergence with

$$\mathbb{D}_{KL}[\pi_{\theta}||\pi_{\text{ref}}](i, t) = \frac{\pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1$$

- the advantage estimate is defined as $\hat{A}_{i,t} := \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$ with $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$.

Remark

- The reverse KL divergence is used, which has the mode-seeking behavior. The usage of k_3 estimator is crucial for GRPO because k_3 guarantee to be non-negative. Otherwise, if one adopts the k_1 estimator, then there could be a possibility to "hack" the training process by making the k_1 estimator arbitrarily small (very negative), then the reward is $\mathcal{J}_{GRPO}(\theta)$ is also maimized. However, this means that π_{θ} is very different from π_{ref} . At the same time, numerical instability is observed during the training dynamics. Because the computation of $\exp \left(\frac{\pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} \right)$ can be unstable.
- In the computation of the KL term, note that $o_i \sim \pi_{\theta_{\text{old}}}$. But the link suggests that o_i should be sampled from π_{θ} to make $\mathbb{D}_{KL}[\pi_{\theta}||\pi_{\text{ref}}]$ an unbiased estimator. In practice, to make these make sense, one has to initialize $\pi_{\theta} := \pi_{\theta_{\text{old}}}$ and take only one step update.
- When optimize the objective function, one of tries to $\min_{\theta} -\mathcal{J}_{GRPO}(\theta)$. One should notice that $-\mathcal{J}_{GRPO}$ can be smaller than 0, meaning the loss can be negative. This could happen when "min" term is larger than the KL term, i.e., when either the two distributions are quite different or the advantage is large.

7 Group Sequence Policy Optimization

In GRPO, the token-level importance sampling weight $\frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$ is applied.

GSPO argues that this is problematic since this weight is based on a single sample $o_{i,t}$ from each next-token distribution $\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})$, it fails to perform the intended distribution-correction role³. Instead, it introduces high-variance noise into the training gradients, which accumulates over long sequences and is exacerbated by the clipping mechanism.

Therefore, GSPO proposes to use the sequence-level importance weight $\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}$. Therefore, the loss function becomes (ignoring the KL regularization term)

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left[\frac{1}{G} \sum_{i=1}^G \min \left[s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right] \right],$$

where $s_i(\theta) = \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} \right)^{1/|o_i|} = \exp \left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \left\{ \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \right\} \right)$, and $\hat{A}_i = \frac{r(x, o_i) - \text{mean}(\{r(x, o_j)\}_{j=1}^G)}{\text{std}(\{r(x, o_j)\}_{j=1}^G)}$.

Therefore, GSPO applies clipping to entire responses instead of individual tokens to exclude the overly "off-policy" samples from gradient estimation, which matches both the sequence-level rewarding and optimization. Also, note that the clipping ranges in GSPO and in previous algorithms (e.g., GRPO) typically differ in order of magnitude due to the distinct definitions of importance ratios.

In scenarios like multi-turn RL, we may desire a finer-grained advantage adjustment than the sequence level.

$$\mathcal{J}_{\text{GSPO-token}}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[s_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(s_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t} \right] \right\} \right],$$

where $s_{i,t}(\theta) = \text{sg}[s_i(\theta)] \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\text{sg}[\pi_\theta(o_{i,t}|q, o_{i,<t})]}$ and sg is the stop gradient operator, so numerically, $s_{i,t}(\theta) = s_i(\theta)$.

To see more clearly about the gradient of the GSPO, GSPO-token, and GRPO, we can compute (clipping is omitted for brevity)

$$\begin{aligned} \nabla_\theta \mathcal{J}_{\text{GSPO}}(\theta) &= \nabla_\theta \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \hat{A}_i \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \hat{A}_i \cdot \nabla_\theta \log s_i(\theta) \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \left(\frac{\pi_\theta(o_i|x)}{\pi_{\theta_{\text{old}}}(o_i|x)} \right)^{\frac{1}{|o_i|}} \hat{A}_i \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \nabla_\theta \log \pi_\theta(o_{i,t} | x, o_{i,<t}) \right] \\ \nabla_\theta \mathcal{J}_{\text{GSPO-token}}(\theta) &= \nabla_\theta \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} s_{i,t}(\theta) \hat{A}_{i,t} \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \hat{A}_{i,t} \frac{\nabla_\theta \pi_\theta(o_{i,t} | x, o_{i,<t})}{\pi_\theta(o_{i,t} | x, o_{i,<t})} \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \left(\frac{\pi_\theta(o_i|x)}{\pi_{\theta_{\text{old}}}(o_i|x)} \right)^{\frac{1}{|o_i|}} \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \hat{A}_{i,t} \nabla_\theta \log \pi_\theta(o_{i,t} | x, o_{i,<t}) \right] \\ \nabla_\theta \mathcal{J}_{\text{GRPO}}(\theta) &= \nabla_\theta \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} w_{i,t}(\theta) \hat{A}_{i,t} \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \hat{A}_i \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \frac{\pi_\theta(o_{i,t} | x, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | x, o_{i,<t})} \nabla_\theta \log \pi_\theta(o_{i,t} | x, o_{i,<t}) \right] \end{aligned}$$

³Recall that Importance sampling needs multiple samples from the same distribution to reduce variance.

Therefore, the fundamental distinction between GSPO and GRPO lies in how they weight the gradients of the log likelihoods of tokens. In GRPO, the tokens are weighted according to their respective "importance weight". In contrast, GSPO weights all the tokens in a response equally, eliminating this instability factor of GRPO. GSPO-token and GSPO are numerically identical in the optimization objective, clipping condition, and theoretical gradient when we set the advantages of all the tokens in the response o_i to the same value (i.e., $\hat{A}_{i,t} = \hat{A}_i$), while GSPO-token enjoys the higher flexibility of adjusting the advantages per token.

$$r_{i,t} = \begin{cases} r_{i,t}^{\text{re}} + \gamma^{T_i-t} R_{\text{out},i}, & R_{\text{out},i} = +1, \\ \text{sim}_{i^*,t} \cdot r_{i^*,t}^{\text{re}} + \gamma^{T_i-t} R_{\text{out},i}, & R_{\text{out},i} = -1. \end{cases} \quad (17)$$

8 Clipped Importance Sampling Policy Optimization

Clipped Importance Sampling Policy Optimization (CISPO (Chen et al., 2025)) propped in MiniMax-M1 optimizes a policy-gradient objective using a clipped importance sampling ratio as a stop-gradient weight. The CISPO objective is defined as

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} | q, o_{i,<t}) \right].$$

Here, $\hat{A}_{i,t}$ is the estimated advantage for token $o_{i,t}$, and $\text{sg}(\cdot)$ denotes the stop-gradient operation. Therefore, $\text{sg}(\hat{r}_{i,t}(\theta))$ is treated as a constant when taking gradients with respect to θ . The clipped importance sampling ratio is

$$\hat{r}_{i,t}(\theta) = \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}),$$

where the token-level importance sampling ratio is

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}.$$

Equivalently,

$$\hat{r}_{i,t}(\theta) = \begin{cases} 1 - \epsilon_{\text{low}}, & r_{i,t}(\theta) < 1 - \epsilon_{\text{low}}, \\ r_{i,t}(\theta), & 1 - \epsilon_{\text{low}} \leq r_{i,t}(\theta) \leq 1 + \epsilon_{\text{high}}, \\ 1 + \epsilon_{\text{high}}, & r_{i,t}(\theta) > 1 + \epsilon_{\text{high}}. \end{cases}$$

Thus, CISPO does not clip the surrogate objective itself. Instead, it clips the token-level importance sampling ratio and uses the clipped ratio as a stop-gradient coefficient in front of the log-probability term.

8.1 Gradient Difference between PPO and CISPO

The main difference between PPO and CISPO is how clipping affects the policy gradient. For PPO, the clipped surrogate objective at token (i, t) can be written as

$$L_{i,t}^{\text{PPO}}(\theta) = \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right),$$

where

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}.$$

Consider the case

$$\hat{A}_{i,t} > 0 \quad \text{and} \quad r_{i,t}(\theta) > 1 + \epsilon.$$

Then the clipped PPO objective becomes

$$L_{i,t}^{\text{PPO}}(\theta) = (1 + \epsilon) \hat{A}_{i,t}.$$

This term is constant with respect to θ , and therefore

$$\nabla_{\theta} L_{i,t}^{\text{PPO}}(\theta) = 0.$$

Thus, when the probability of a positive-advantage token has already increased beyond the clipping range, PPO removes the gradient contribution from this token.

Similarly, if

$$\hat{A}_{i,t} < 0 \quad \text{and} \quad r_{i,t}(\theta) < 1 - \epsilon,$$

then the clipped PPO objective becomes

$$L_{i,t}^{\text{PPO}}(\theta) = (1 - \epsilon) \hat{A}_{i,t},$$

which is also constant with respect to θ . Hence,

$$\nabla_{\theta} L_{i,t}^{\text{PPO}}(\theta) = 0.$$

In this case, PPO removes the gradient contribution when the probability of a negative-advantage token has already decreased beyond the clipping range.

By contrast, CISPO uses the clipped importance sampling ratio as a stop-gradient coefficient:

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) \right].$$

Since

$$\text{sg}(\hat{r}_{i,t}(\theta))$$

is treated as a constant during backpropagation, the gradient of the CISPO objective is

$$\nabla_{\theta} \mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \nabla_{\theta} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) \right].$$

Therefore, even when the ratio $r_{i,t}(\theta)$ falls outside the clipping interval, CISPO does not make the gradient of that token vanish. Instead, the clipped ratio $\hat{r}_{i,t}(\theta)$ controls the magnitude of the gradient, while the token still contributes a policy-gradient signal through

$$\nabla_{\theta} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}).$$

In short, PPO clips the surrogate objective, which can cause clipped tokens to have zero gradient. CISPO clips the importance sampling ratio and uses it as a stop-gradient weight, so clipped tokens can still contribute to the policy gradient.

Part III

Asynchronous RL

For agentic workloads, especially long-horizon tasks, rollout lengths are highly variable, so short trajectories finish quickly while long ones become stragglers.

Asynchronous RL algorithms are proposed to mitigate this imbalanced generation overhead by consuming rollouts continuously as they arrive, improving utilization and wall-clock efficiency. However, asynchrony introduces one challenge, where each trajectory can be generated by multiple versions of the old rollout model, which leads to more unpredictable and severe off-policy, and thus harms the training stability.

Recall that in the Standard PPO, the objective is defined as

$$\max_{\theta} \mathcal{J}_{\text{PPO-clip}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \min \left\{ \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_{\phi}(s_t, a_t), \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\phi}(s_t, a_t) \right\} \right],$$

controls policy update size via the ratio $\frac{\pi_{\theta}}{\pi_{\theta_{\text{old}}}}$, where $\pi_{\theta_{\text{old}}}$ is assumed to be both the *proximal policy* (the anchor policy for PPO clipping) and the *behavior policy* (the policy that collected the data); both terms are defined in (Hilton et al., 2022), which was primarily proposed to achieve batch size-invariance for PPO.

The key idea is to decouple these two roles. Let $\pi_{\theta_{\text{prox}}}$ denote the proximal policy that anchors the clipping region, and let $\pi_{\theta_{\text{beh}}}$ denote the behavior policy that collected the trajectories. Then a three-policy version of PPO-Clip can be written as

$$\max_{\theta} \mathcal{J}_{\text{PPO-Decoupled}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{beh}}}} \left[\sum_{t=0}^{T-1} w_t \min \left\{ r_t(\theta) \hat{A}_{\phi}(s_t, a_t), \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\phi}(s_t, a_t) \right\} \right],$$

where

$$w_t := \frac{\pi_{\theta_{\text{prox}}}(a_t|s_t)}{\pi_{\theta_{\text{beh}}}(a_t|s_t)}, \quad r_t(\theta) := \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{prox}}}(a_t|s_t)}.$$

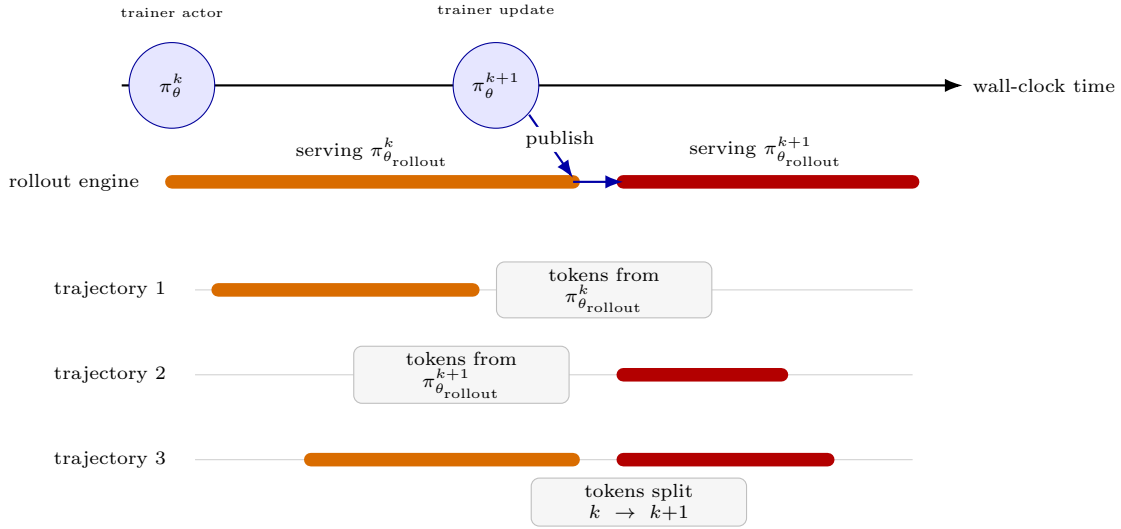
Here, w_t is the importance sampling correction from the behavior policy to the proximal policy. During one policy update, $\pi_{\theta_{\text{prox}}}$ and $\pi_{\theta_{\text{beh}}}$ are fixed, so w_t is treated as a constant with respect to θ . In contrast, $r_t(\theta)$ is the PPO ratio used to control how far the current policy π_{θ} moves away from the proximal policy $\pi_{\theta_{\text{prox}}}$.

Remark

- If $\pi_{\theta_{\text{beh}}} = \pi_{\theta_{\text{prox}}} = \pi_{\theta_{\text{old}}}$, then $w_t = 1$ and the objective reduces to the standard PPO-Clip objective.
- The behavior policy $\pi_{\theta_{\text{beh}}}$ can be stale or generated by a different worker, while the update size is still controlled relative to $\pi_{\theta_{\text{prox}}}$.
- The clipping mechanism is preserved because the clipped ratio is $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{prox}}}(a_t|s_t)$, not the behavior correction w_t .

For the asynchronous discussion, it is clearer to write the proximal policy as $\pi_{\theta_{\text{old}}}$ and the behavior policy as $\pi_{\theta_{\text{rollout}}}$. The policy $\pi_{\theta_{\text{old}}}$ is the frozen PPO anchor used in the clipping ratio, while $\pi_{\theta_{\text{rollout}}}$ is the model that actually generated the sampled token or action. For more details, see the veRL discussion and the Slime reference implementation.

Consequently, the data consumed by a trainer update may not have a single behavior policy. Some trajectories may have been generated before the rollout engine refreshed, while later trajectories may use the refreshed rollout weights; in a long rollout, even different tokens in the same trajectory can correspond to different rollout snapshots. See Figure 2 for a visualization.



A training window can contain rollouts from different rollout-engine snapshots.

The stored rollout log-probability identifies the behavior policy for each sampled token.

Figure 2: Policy lag comes from the time gap between the trainer actor update and the rollout engine refresh. During this gap, the trainer has advanced to π_{θ}^{k+1} while the rollout engine may still generate tokens with $\pi_{\theta_{\text{rollout}}}^k$; after refresh, later tokens are generated with $\pi_{\theta_{\text{rollout}}}^{k+1}$. A trajectory that spans the refresh can therefore contain tokens from both rollout snapshots.

Thus, exact off-policy correction requires the behavior probability for the particular sampled token/action, namely the rollout-time value $\pi_{\theta_{\text{rollout}}}(a_t|s_t)$ used when that token/action was sampled. In practice, this means storing the rollout log-probability together with each generated token. If these log-probabilities are not stored, then one would need to keep enough rollout-policy checkpoints to recompute them later. The required history is therefore a history of behavior probabilities, or equivalently rollout-policy snapshots. In the long horizon setting, maintaining the history is not feasible in practice.

In this part, we discuss various asynchronous RL algorithms target to address various issues arised in the asynchronous RL.

9 SAO

9.1 Algorithm

To address the policy lag issue, SAO (Hou et al., 2026) proposes several modifications over the PPO-Clip objective to reach the following objective:

$$\max_{\theta} \mathcal{J}_{\text{SAO}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{rollout}}}} \left\{ \sum_{t=0}^{T-1} \left[\text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{rollout}}}(a_t | s_t)}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{\phi}(s_t, a_t) \right] \log \pi_{\theta}(a_t | s_t) \right\}, \quad (18)$$

where (18) is written in REINFORCE form (7): the clipped importance ratio times the advantage is treated as a detached weight, and the explicit $\log \pi_{\theta}(a_t | s_t)$ term provides the policy-gradient direction.

Remark

1. SAO drops the proximal policy $\pi_{\theta_{\text{old}}}$ and uses the behavior policy $\pi_{\theta_{\text{rollout}}}$ for importance sampling:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{rollout}}}(a_t | s_t)}.$$

This also avoids recomputing probabilities under $\pi_{\theta_{\text{old}}}$ by reusing the log-probabilities generated during rollout.

2. SAO applies a different trust region by clipping the rollout ratio to $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$. As shown in Figure 3, PPO clips by asking “Has this sample already moved too far in the direction that appears beneficial?”, while SAO clips by asking “Is the current policy still sufficiently close to the policy that generated this token for this sample to be trustworthy?”.
 - **PPO’s asymmetric construction is useful** when the advantage estimation is reliable, the data is fresh, the large ratio arose from parameter coupling or prior minibatch updates, and we want the sample to help undo an undesirable policy shift. For example, if $A = -1$ and $r = 10$, PPO-Clip keeps the directionally corrective term $rA = -10$, which pushes the sampled action’s probability down, while SAO clips it. Similarly, if $A = 1$ and $r = 0.1$, PPO keeps the corrective gradient, while SAO clips it. Because only one side is saturated for each advantage sign, PPO generally uses more of the batch and supports repeated minibatch optimization over collected data. However, PPO’s retained corrective gradients can become dangerous in asynchronous or heavily off-policy training.
 - **SAO’s symmetric construction is helpful** in asynchronous or heavily off-policy training. For $A < 0$ and $r \gg 1$, the factor rA can dominate a minibatch. That may be acceptable if A is accurate and the sample is still relevant, but it can be destructive when the sample was generated by a substantially older actor, the trajectory context is highly off-policy, or one extreme token dominates the aggregate gradient. SAO bounds the importance ratio regardless of the advantage sign, giving direct control over this source of gradient heavy tails.

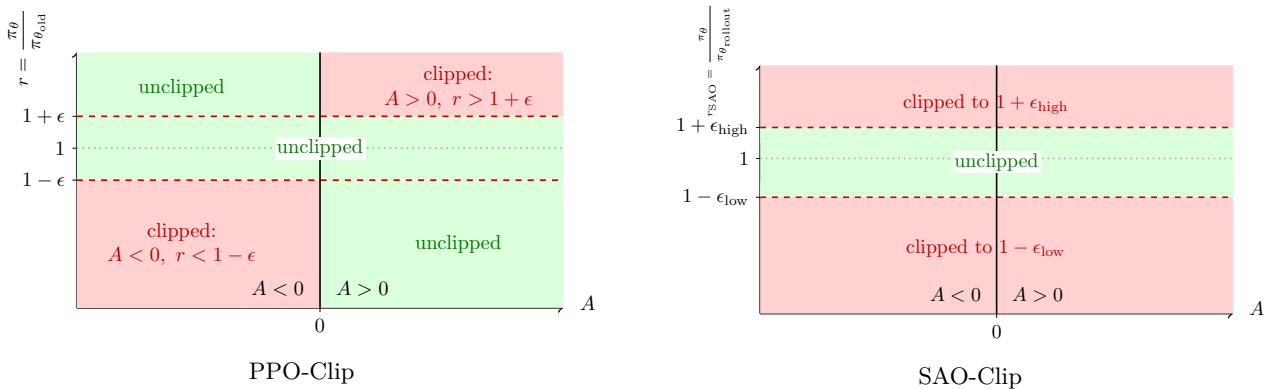


Figure 3: PPO-Clip and SAO-Clip trust regions in the (A, r) plane. PPO clips only in the regions selected by the clipped surrogate objective, while SAO directly clips the rollout-ratio $r = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{rollout}}}(a_t | s_t)}$ to $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$.

3. The strict two-sided gate has costs. First, once a token is outside the band, it gives no direct gradient, even when its advantage would have produced a restoring update. This is a distinction between filtering and constraining: SAO filters data; it does not project or roll the policy back into a feasible region. Second,

importance sampling is unbiased only without truncation under the usual assumptions. PPO introduces bias through one-sided clipping, while SAO introduces more bias by dropping both tails. In particular, dropping a negative-advantage outlier can make the observed surrogate look artificially favorable because a large negative contribution has disappeared. Therefore, SAO’s gated objective does not have PPO’s uniform pointwise pessimism property. Third, clipping reduces the effective batch size: if many tokens leave the band, the nominal batch size may remain large while the number of contributing tokens collapses.

4. The paper does not specify every detail of the advantage estimator $\hat{A}_\phi$. We sketch a consistent token-level interpretation below.

9.2 Guessed Advantage Estimator

The main ambiguity is how to align token-level policy decisions, value estimates, rewards, and observations in a serialized agent trajectory. The following convention uses a pre-token state: the value assigned to a generated token evaluates the prefix before that token is sampled.

Serialized trajectory and physical token positions. Let the complete serialized trajectory be

$$z_{0:T-1} = [\text{prompt}, a_0, o_0, a_1, o_1, \dots],$$

where the i -th action span is

$$a_i = (a_{i,0}, a_{i,1}, \dots, a_{i,N_i}),$$

and o_i is the environment observation returned after action span a_i . Let $p_{i,j}$ be the physical position of action token $a_{i,j}$ in the full serialized sequence:

$$a_{i,j} = z_{p_{i,j}}.$$

The positions $p_{i,j}$ are not consecutive across action boundaries because observation tokens occur between action spans.

State associated with an action token. Define the decision state for action token $a_{i,j}$ as the full prefix strictly before that token:

$$S_{i,j} := z_{<p_{i,j}} = (z_0, \dots, z_{p_{i,j}-1}).$$

Then the token is sampled according to

$$a_{i,j} \sim \pi_\theta(\cdot \mid S_{i,j}).$$

The corresponding scalar critic prediction is

$$v_{i,j} := V_\phi(S_{i,j}).$$

We use lowercase $v_{i,j}$ for the scalar prediction and reserve $V_\phi(\cdot)$ for the value function itself. Under this convention, the actor and critic are aligned:

$$\pi_\theta(\cdot \mid S_{i,j}) \quad \text{and} \quad V_\phi(S_{i,j})$$

both operate on the same pre-token state.

Transformer hidden-state alignment. Suppose a causal transformer produces

$$h_t = H_\phi(z_{\leq t}),$$

so h_t represents the prefix ending at token z_t and predicts the next token z_{t+1} . Since $a_{i,j} = z_{p_{i,j}}$, the state before choosing $a_{i,j}$ is represented by $h_{p_{i,j}-1}$. Therefore,

$$\log \pi_\theta(a_{i,j} \mid S_{i,j}) = [\log \text{softmax}(W_\pi h_{p_{i,j}-1})]_{a_{i,j}}, \quad (19)$$

$$v_{i,j} = V_\phi(S_{i,j}) = w_V^\top h_{p_{i,j}-1}. \quad (20)$$

The hidden state is merely the implementation representation of the mathematical state $S_{i,j}$.

Interior of an action span. For an interior action token $j < N_i$, the next policy decision is the next token in the same action span. Therefore,

$$S_{i,j+1} = S_{i,j} \oplus a_{i,j},$$

where $\oplus$ denotes sequence concatenation. The TD residual associated with choosing $a_{i,j}$ is

$$\delta_{i,j} := \rho_{i,j} + \gamma v_{i,j+1} - v_{i,j}, \quad j < N_i.$$

Here $\rho_{i,j}$ is the reward attributed to that token. Usually $\rho_{i,j} = 0$ for interior tokens unless token-level shaping rewards are used. The GAE recursion is

$$A_{i,j} := \delta_{i,j} + \gamma \lambda A_{i,j+1}.$$

Action–observation boundary. For the final token a_{i,N_i} , the next policy decision is not an observation token; it is the first token of the next action, $a_{i+1,0}$. The corresponding next decision state is

$$S_{i+1,0} = S_{i,N_i} \oplus a_{i,N_i} \oplus o_i.$$

This state includes the last token of action i , the complete environment observation o_i , and any role markers or formatting delimiters before the next assistant generation. The boundary TD residual is therefore

$$\delta_{i,N_i} := R_i + \gamma v_{i+1,0} - v_{i,N_i},$$

where

$$v_{i,N_i} := V_\phi(S_{i,N_i}), \quad v_{i+1,0} := V_\phi(S_{i+1,0}).$$

The boundary GAE recursion is

$$A_{i,N_i} := \delta_{i,N_i} + \gamma \lambda A_{i+1,0}.$$

This is the precise version of writing values at the last token of one action and the first token of the next action. Under the pre-token convention, those values are $V_\phi(S_{i,N_i})$ and $V_\phi(S_{i+1,0})$. The intended connection is between the last model-generated token of one action and the first model-generated token of the next action, while observation-token positions are omitted from GAE.

Concrete example. Consider the serialized sequence

$$[q_0, q_1, a_{0,0}, a_{0,1}, o_{0,0}, o_{0,1}, o_{0,2}, a_{1,0}, a_{1,1}].$$

The relevant pre-token states are

$$\begin{aligned} S_{0,0} &= [q_0, q_1], \\ S_{0,1} &= [q_0, q_1, a_{0,0}], \\ S_{1,0} &= [q_0, q_1, a_{0,0}, a_{0,1}, o_{0,0}, o_{0,1}, o_{0,2}]. \end{aligned}$$

Consequently,

$$\begin{aligned} v_{0,0} &= V_\phi([q_0, q_1]), \\ v_{0,1} &= V_\phi([q_0, q_1, a_{0,0}]), \\ v_{1,0} &= V_\phi([q_0, q_1, a_{0,0}, a_{0,1}, o_{0,0}, o_{0,1}, o_{0,2}]). \end{aligned}$$

For the first action token,

$$\delta_{0,0} = \rho_{0,0} + \gamma v_{0,1} - v_{0,0}.$$

For the last token of action 0,

$$\delta_{0,1} = R_0 + \gamma v_{1,0} - v_{0,1}.$$

No values are evaluated at $o_{0,0}, o_{0,1}, o_{0,2}$ for the purpose of GAE, but the entire observation is still included in the state used to compute $v_{1,0}$.

Implementation warning about tensor indexing. A value-model forward pass often returns one scalar per physical input position:

$$\tilde{v}_t = w_V^\top h_t.$$

However, $\tilde{v}_t$ is the value of the prefix $z_{\leq t}$. Therefore, for the action token at physical position $p_{i,j}$, the pre-token value is

$$v_{i,j} = \tilde{v}_{p_{i,j}-1}, \quad \text{not } \tilde{v}_{p_{i,j}}.$$

In tensor code, this typically corresponds to shifting values in the same way that logits are shifted against next-token labels:

```
# hidden[:, t] represents input_ids[:, :t+1]
logits = policy_head(hidden[:, :-1])
state_values = value_head(hidden[:, :-1])

# aligned with labels/input tokens at positions 1, ..., T-1
next_tokens = input_ids[:, 1:]
```

Some implementations may store the shifted value afterward at the action token’s array position. That is fine. What matters is the semantics: the value assigned to action token $a_{i,j}$ must evaluate the prefix before $a_{i,j}$. Using $h_{p_{i,j}}$ instead would define a post-token value $V_\phi(z_{\leq p_{i,j}})$. That convention can also be made valid, but all TD equations must then be re-indexed consistently. Mixing a pre-token policy log-probability with a post-token value under the same symbol is exactly the ambiguity we want to avoid.

9.3 Practical Considerations

What to do?

- **Faster Value Update than Policy.** The authors attribute primary source of instability in single-rollout RL is the interdependence between the policy and the value function. If the value model V_ϕ is inaccurate, the advantage estimates $\hat{A}_\phi(s_t, a_t)$ become noisy, leading to destructive policy updates. Specifically, for every single gradient update applied to the policy π_θ , we enforce K updates to the value network V_ϕ (default $K = 2$).
- **Stabilizing Value Model Training via Parameter Freezing.** Instability of value model training, where the gradient norms of the value model are significantly larger than the corresponding policy model. Further decomposition shows that this instability originates primarily from the Full Attention layers. As a result, the authors freeze the parameters of the Full Attention layers during value model training.

What to monitor? A strong hybrid is often:

gross-staleness gate+PPO or soft rollback within the gate+explicit KL/TV monitoring.

For diagnosing whether the extra SAO masking is helping, the most informative measurements are not merely the aggregate clip fraction. Track the four A-by-r quadrants separately, the accepted-token fraction by actor version, importance-weight effective sample size, log-ratio quantiles, and the gradient norm contributed by the $A < 0, r > u$ quadrant. That last quadrant is where SAO is most likely to gain stability and where it sacrifices the strongest potentially corrective PPO signal.

References

- Chen, A., Li, A., Gong, B., Jiang, B., Fei, B., Yang, B., Shan, B., Yu, C., Wang, C., Zhu, C., Xiao, C., Du, C., Zhang, C., Qiao, C., Zhang, C., Du, C., Guo, C., Chen, D., Ding, D., Sun, D., Li, D., Jiao, E., Zhou, H., Zhang, H., Ding, H., Sun, H., Feng, H., Cai, H., Zhu, H., Sun, J., Zhuang, J., Cai, J., Song, J., Zhu, J., Li, J., Tian, J., Liu, J., Xu, J., Yan, J., Liu, J., He, J., Feng, K., Yang, K., Xiao, K., Han, L., Wang, L., Yu, L., Feng, L., Li, L., Zheng, L., Du, L., Yang, L., Zeng, L., Yu, M., Tao, M., Chi, M., Zhang, M., Lin, M., Hu, N., Di, N., Gao, P., Li, P., Zhao, P., Ren, Q., Xu, Q., Li, Q., Wang, Q., Tian, R., Leng, R., Chen, S., Chen, S., Shi, S., Weng, S., Guan, S., Yu, S., Li, S., Zhu, S., Li, T., Cai, T., Liang, T., Cheng, W., Kong, W., Li, W., Chen, X., Song, X., Luo, X., Su, X., Li, X., Han, X., Hou, X., Lu, X., Zou, X., Shen, X., Gong, Y., Ma, Y., Wang, Y., Shi, Y., Zhong, Y., Duan, Y., Fu, Y., Hu, Y., Gao, Y., Fan, Y., Yang, Y., Li, Y., Hu, Y., Huang, Y., Li, Y., Xu, Y., Mao, Y., Shi, Y., Wenren, Y., Li, Z., Li, Z., Tian, Z., Zhu, Z., Fan, Z., Wu, Z., Xu, Z., Yu, Z., Lyu, Z., Jiang, Z., Gao, Z., Wu, Z., Song, Z., and Sun, Z. (2025). Minimax-m1: Scaling test-time compute efficiently with lightning attention.
- Hilton, J., Cobbe, K., and Schulman, J. (2022). Batch size-invariance for policy optimization.
- Hou, Z., Li, Y., Tang, J., and Dong, Y. (2026). Single-rollout asynchronous optimization for agentic reinforcement learning. [arXiv preprint arXiv:2607.07508](#).
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. [Advances in neural information processing systems](#), 35:27730–27744.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. [arXiv preprint arXiv:1506.02438](#).
- Xie, T., Gao, Z., Ren, Q., Luo, H., Hong, Y., Dai, B., Zhou, J., Qiu, K., Wu, Z., and Luo, C. (2025). Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. [arXiv preprint arXiv:2502.14768](#).